DevDocs — File Path & Change Detection in V1

1. Why DevDocs Stores the File Path

DevDocs should store the **full relative file path** of every file.

For example:

```
backend/auth.py
```

and:

```
backend/security/auth.py
```

are considered **different files**, even if:

- They have the same filename: `auth.py`
- Their contents are identical
- Their SHA-256 hashes are identical

The path tells DevDocs **where the file exists inside the project**.

Therefore, DevDocs should store both:

```
file_path
file_hash
```

---

2. What Does the File Path Tell Us?

The file path answers:

> **Which file are we talking about?**

For example:

```
backend/auth.py
```

identifies one specific file.

While:

```
backend/security/auth.py
```

identifies another file.

The filename alone is not enough.

Both of these have the filename:

```
auth.py
```

but they are different files because their paths are different.

---

3. What Does the Hash Tell Us?

The file hash answers:

> **Did the content of this file change?**

For example:

```
backend/auth.py
hash = ABC123
```

If the user uploads the project again and DevDocs sees:

```
backend/auth.py
hash = ABC123
```

then the content has not changed.

If it sees:

```
backend/auth.py
hash = XYZ789
```

then the content has changed.

---

4. Path and Hash Work Together

DevDocs uses:

```
file_path + file_hash
```

to perform V1 change detection.

Think of it as:

```
Path
 ↓
Which file?

Hash
 ↓
Did that file's content change?
```

So:

```
file_path = backend/auth.py
file_hash = ABC123
```

identifies the file's location and its current content fingerprint.

---

5. Important: Same Name Does Not Mean Same File

Consider:

```
backend/auth.py
```

and:

```
backend/security/auth.py
```

Both have the same filename:

```
auth.py
```

But they are different files.

Therefore:

```
Filename alone ❌
```

is not sufficient.

DevDocs should use:

```
Relative path ✅
```

---

6. The Five V1 Change-Detection Conditions

DevDocs should explicitly handle these five situations.

Condition 1 — Same Path + Same Hash

Example

Previous upload:

```
backend/auth.py
hash = ABC123
```

New upload:

```
backend/auth.py
hash = ABC123
```

Comparison:

```
Path → SAME
Hash → SAME
```

Result

```
UNCHANGED
```

Meaning

The file exists in the same location and its contents have not changed.

DevDocs action

Skip:

```
Parsing
Chunking
Embedding
```

The existing chunks and embeddings can continue to be used.

---

7. Condition 2 — Same Path + Different Hash

Example

Previous upload:

```
backend/auth.py
hash = ABC123
```

New upload:

```
backend/auth.py
hash = XYZ789
```

Comparison:

```
Path → SAME
Hash → DIFFERENT
```

Result

```
MODIFIED
```

Meaning

It is the **same file**, because its path is the same, but its contents have changed.

For example:

Old content

```
def login():
    print("Login")
```

New content

```
def login():
    print("User login successful")
```

The filename is still:

```
auth.py
```

The path is still:

```
backend/auth.py
```

But the content changed.

Therefore the hash changed.

DevDocs action

```
Delete old chunks
        ↓
Parse new file
        ↓
Create new chunks
        ↓
Generate new embeddings
        ↓
Store new chunks/embeddings
        ↓
Update file hash
```

---

8. Condition 3 — New Path Appears

Previous project

```
backend/
├── main.py
└── auth.py
```

New project

```
backend/
├── main.py
├── auth.py
└── security.py
```

DevDocs sees:

```
backend/security.py
```

did not exist previously.

Result

```
CREATED
```

DevDocs action

Process the new file:

```
Parse
 ↓
Chunk
 ↓
Embed
 ↓
Store
```

---

9. Condition 4 — Old Path Disappears

Previous project

```
backend/
├── main.py
├── auth.py
└── database.py
```

New project

```
backend/
├── main.py
└── auth.py
```

DevDocs sees:

```
backend/database.py
```

existed before but no longer exists.

Result

```
DELETED
```

DevDocs action

Remove:

```
File metadata
Chunks
Embeddings
```

This is important because otherwise the RAG system could retrieve information from a file that no longer exists in the current project.

---

10. Condition 5 — Different Path + Same Hash

This is the **rename/move case**.

Previous project

```
backend/auth.py
hash = ABC123
```

New project

```
backend/security/auth.py
hash = ABC123
```

Comparison:

```
Path → DIFFERENT
Hash → SAME
```

The content is identical.

The likely explanation is that the file was moved:

```
backend/auth.py
        ↓
backend/security/auth.py
```

or renamed/moved to another location.

V1 Behavior

DevDocs does **not** need to detect the rename/move intelligently in V1.

It can treat this as:

```
Old:
backend/auth.py
        ↓
DELETED
```

and:

```
New:
backend/security/auth.py
```

```
        ↓
CREATED
```

Therefore, DevDocs will process the file again.

This is acceptable for V1.

---

11. Why Is Rename Detection Not Required in V1?

Detecting a rename is an optimization.

The simplest reliable V1 logic is:

```
Compare old paths with new paths
        ↓
Compare hashes for matching paths
```

If a path disappears:

```
Deleted
```

If a new path appears:

```
Created
```

DevDocs does not need to determine whether those two events were actually caused by a rename.

This keeps the implementation simple.

---

12. Future Rename Optimization

A future version could detect this:

```
OLD

backend/auth.py
hash = ABC123
```

and:

```
NEW

backend/security/auth.py
hash = ABC123
```

and conclude:

> The old file disappeared and a new file appeared with exactly the same content. This is probably a move or rename.

Instead of:

```
Delete old file
+
Process new file
```

DevDocs could potentially:

```
Reuse existing chunks
Reuse existing embeddings
Update the file path
```

This avoids unnecessary re-processing.

But this is **not required for V1**.

---

13. Complete V1 Rules

The five rules can be summarized as:

Previous State	New State	Result
Same path + same hash	Same path + same hash	**Unchanged**
Same path + different hash	Same path + different hash	**Modified**
Path did not exist	New path appears	**Created**
Old path disappears	Path no longer exists	**Deleted**
Old path + same hash	Different new path + same hash	**Deleted + Created**

---

14. Simple Mental Model

Remember these two questions:

Question 1

> **Where is the file?**

Answer:

```
file_path
```

Question 2

> **Did the content change?**

Answer:

```
file_hash
```

Therefore:

```
Path = file location
Hash = content fingerprint
```

Together:

```
file_path + file_hash
```

allow DevDocs to perform simple and reliable V1 change detection.

---

15. Example With a Real Project

First upload

```
my-project/
│
├── backend/
│   ├── auth.py
│   └── database.py
```

```
│
└── README.md
```

DevDocs stores:

```
backend/auth.py
hash = A

backend/database.py
hash = B

README.md
hash = C
```

Second upload

The user:

- Changes `auth.py`
- Deletes `database.py`
- Adds `security.py`
- Doesn't change `README.md`

New project:

```
my-project/
│
├── backend/
│   ├── auth.py
│   └── security.py
│
└── README.md
```

DevDocs compares:

```
backend/auth.py
Old hash = A
New hash = X
→ MODIFIED
```

```
backend/database.py
Old path exists
New path missing
→ DELETED
```

```
backend/security.py
Old path missing
New path exists
→ CREATED
```

```
README.md
Old hash = C
New hash = C
→ UNCHANGED
```

So DevDocs only needs to re-process:

```
backend/auth.py
backend/security.py
```

and remove:

```
backend/database.py
```

while leaving:

```
README.md
```

untouched.

---

16. Core V1 Principle

> **The path identifies the file; the hash identifies whether its content changed.**

And:

> **Same path + same hash = skip processing.**
>
> **Same path + different hash = re-process.**
>
> **New path = process.**
>
> **Missing old path = remove.**
>
> **Different path + same hash = treat as delete + create in V1.**

This is the complete V1 change-detection model for DevDocs.